HDOC Day-66

Multiplication of Two Square Matrices Using Recursion in C

Aim

To write a C program using recursion, user-defined functions, two-dimensional arrays, and conditional statements to multiply two square matrices without iterative loops for the multiplication logic.

Algorithm

1. Read the order n of the two square matrices and verify that 1 <= n <= 10.
2. Read the elements of matrices A and B.
3. Initialize the result matrix C to zero.
4. Call multiplyMatrices(n, A, B, C, 0, 0, 0).
5. If i == n, stop recursion because all rows have been processed.
6. If j == n, recursively advance to the next row (i + 1, 0, 0).
7. If k == n, recursively advance to the next column (i, j + 1, 0).
8. Otherwise add A[i][k] * B[k][j] to C[i][j], then recurse with k + 1.
9. Display the resultant matrix and stop.

C Program

```
#include <stdio.h>
#define MAX 10

void multiplyMatrices(int n, int A[n][n], int B[n][n],
                      int C[n][n], int i, int j, int k);

int main()
{
    int n, i, j;
    printf("Enter the order of square matrices: ");
    scanf("%d", &n);

    if (n <= 0 || n > MAX)
    {
        printf("Invalid matrix order!\n");
        return 0;
    }

    int A[n][n], B[n][n], C[n][n];
    printf("Enter elements of Matrix A:\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            scanf("%d", &A[i][j]);

    printf("Enter elements of Matrix B:\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            scanf("%d", &B[i][j]);

    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            C[i][j] = 0;

    multiplyMatrices(n, A, B, C, 0, 0, 0);
    printf("\nResultant Matrix:\n");
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
            printf("%d ", C[i][j]);
        printf("\n");
    }
    return 0;
}

void multiplyMatrices(int n, int A[n][n], int B[n][n],
                      int C[n][n], int i, int j, int k)
{
    if (i == n)
        return;
    if (j == n)
    {
        multiplyMatrices(n, A, B, C, i + 1, 0, 0);
        return;
    }
    if (k == n)
    {
        multiplyMatrices(n, A, B, C, i, j + 1, 0);
        return;
    }
    C[i][j] += A[i][k] * B[k][j];
    multiplyMatrices(n, A, B, C, i, j, k + 1);
}
```

Explanation

Each resultant element is C[i][j] = sum of A[i][k] * B[k][j] for k from 0 to n - 1. The three recursive indices i, j, and k replace the three nested loops normally used for multiplication. Iterative loops are used only for input, initialization, and display.

Worked Example

```
Matrix A:       Matrix B:       Result C:
1 2             5 6             19 22
3 4             7 8             43 50
```

C[0][0] = 1 x 5 + 2 x 7 = 19; C[0][1] = 1 x 6 + 2 x 8 = 22. C[1][0] = 3 x 5 + 4 x 7 = 43; C[1][1] = 3 x 6 + 4 x 8 = 50.

Time and Space Complexity

Time: O(n^3). Auxiliary recursive stack space: O(n^3) in the worst case for this implementation.

Test Case Table

Case	Order	Input description	Expected result
1	2	Standard 2 x 2 matrices	Correct 2 x 2 product
2	1	A = [5], B = [3]	[15]
3	2	A is identity	Result equals B
4	2	B is identity	Result equals A
5	2	A is zero matrix	Zero matrix
6	2	Negative elements	Correct signed product
7	3	Standard 3 x 3 matrices	Correct 3 x 3 product
8	0	Invalid order	Error message
9	-2	Invalid order	Error message
10	11	Exceeds maximum order	Error message

Sample Outputs

Test Case 1: Two 2 x 2 Matrices

```
Enter the order of square matrices: 2
Enter elements of Matrix A:
1 2
3 4
Enter elements of Matrix B:
5 6
7 8
Resultant Matrix:
19 22
43 50
```

Test Case 2: One 1 x 1 Matrix

```
Enter the order of square matrices: 1
Enter elements of Matrix A:
5
Enter elements of Matrix B:
3
Resultant Matrix:
15
```

Test Case 3: Identity Matrix

```
Enter the order of square matrices: 2
Enter elements of Matrix A:
1 0
0 1
Enter elements of Matrix B:
4 7
2 5
Resultant Matrix:
4 7
2 5
```

Test Case 4: Zero Matrix

```
Enter the order of square matrices: 2
Enter elements of Matrix A:
0 0
0 0
Enter elements of Matrix B:
1 2
3 4
Resultant Matrix:
0 0
0 0
```

Test Case 5: Invalid Order

```
Enter the order of square matrices: 0
Invalid matrix order!
```

Result

The C program multiplies two square matrices using recursive function calls instead of iterative loops for the multiplication logic. It correctly handles valid matrix orders, identity matrices, zero matrices, and signed elements, and displays the resultant matrix.